CS-351: Complier Construction

(Spring 2026)

Lecture 7

Adnan Rashid

Email: adnan.rashid@seecs.edu.pk

Web: https://adrashid.github.io/

Recap

- Types of Compilers
  - Two Pass
  - Three Pass

Outline

- Lexical Analyzer
  - Theory of Automata and FL: A Quick Review
    - Regular Expressions
    - Finite Automata

Lecture's Content

Compiler: Principles, Techniques and Tools (2nd Edition)

by

Alfred V. Aho et al.

Sections 1.1, 3.1, 3.2, 3.3 and 3.4

(Pages 2, 109 and 115 onwards)

Tokens, Patterns and Lexemes

- A token is a pair consisting of:
  - a token name/type: an abstract symbol representing lexical unit e.g., ID, NUM, OP [Affects parsing decisions]
  - an attribute value: [influences translation after parsing]
  - Examples:

    1. <ID, x>
    2. <NUM, 3>
    3. <OP, +>

Tokens, Patterns and Lexemes

- A lexeme is actual sequence of characters from the source program
  - Matches the pattern for a token and is identified by the lexical analyzer as an instance of a token
  - Examples:
    1. if
    2. else
    3. while

Tokens, Patterns and Lexemes

- A pattern is a description of the form that the lexeme of a token may take (A rule describing what form a lexeme of a token can take)
  - In the case of a keyword as a token, the pattern is just the sequence of characters that form the keyword (Fixed strings if, else)
  - For identifier and some other tokens, the pattern is more complex structure that is matched by many strings (complex pattern like letter (letter | digit)*)

Example: Tokens, Patterns/ Description and Lexemes

TOKEN	INFORMAL DESCRIPTION	SAMPLE LEXEMES
if	characters `i`, `f`	`if`
else	characters `e`, `l`, `s`, `e`	`else`
comparison	`<` or `>` or `<=` or `>=` or `==` or `!=`	`<=`, `!=`
id	letter followed by letters and digits	`pi`, `score`, `D2`
number	any numeric constant	`3.14159`, `0`, `6.02e23`
literal	anything but `"`, surrounded by `"`'s	`"core dumped"`

Source: Dragon book, by Alfred Aho et al. pp-112

Goals of Lexical Analysis (Scanners)

- Partition the program into lexemes
- Classify each lexeme into a token
- Ignore irrelevant characters like:
  - spaces
  - tabs
  - comments

Tokens: Common Types in a program

- Identifiers: x y11 maxsize
- Keywords: if else while for
- Integers: 2 1000 -44
- Floats: 2.0 0.0034 1e5
- Symbols: ( ) + * / { } < > ==
- Strings: "enter x" "error"

Tokens: Principle of Longest Match

- In most languages, the scanner should pick the longest possible string to make up the next token if there is a choice

- Example:

  return maybe != iffy;

  should be recognized as 5 tokens

<RETURN>	**<ID, maybe>**	**<NEQ>**	**<ID, iffy>**	**<SCOLON>**

  i.e., != is one token, not two, "iffy" is an ID, not IF followed by <ID, fy>

Tokens: Principle of Longest Match

- Example 2:

  if iffy should be recognized as 2 tokens **<IF>** **<ID, iffy>**

  - Assume the language has:
    - Keyword: if
    - Identifier: letter (letter | digit)*
  - Assume Scanner starts at the first character

  if iffy

  Possible matches:
  - if -> Keyword
  - i -> identifier (but shorter)
  - Longest valid match = if

  Token = **<IF>**

Tokens: Principle of Longest Match

- Example 2:

  if iffy should be recognized as 2 tokens **<IF>** **<ID, iffy>**

  - Assume the language has:
    - Keyword: if
    - Identifier: letter (letter | digit)*
  - Assume Scanner starts at the first character

  if iffy

Possible matches:
- if -> Keyword (matches first two characters)
- iffy -> identifier (matches the entire string)
- Longest valid match = iffy

Token = **<ID, iffy>**

Token Generation: Example

- Tokens generated for the following C program

```
float limitedSquare(x) float x {
    /* returns x-squared, but never more than 100 */
    return (x<=-10.0||x>=10.0)?100:x*x;
}
```

<float> <id, limitedSquare> <(> <id, x> <)> <float> <id, x> <{>
<return> <(> <id, x> <op,"<="> <num, -10.0> <op, "||"> <id, x>
<op, ">="> <num, 10.0> <)> <op, "?"> <num, 100> <op, ":"> <id,
x> <op, "*"> <id, x> <;>
<}>

Theory of Automata and FL: A Quick Review

- Alphabet: a finite set of symbols

  Binary Alphabet: $\Sigma = \{0,1\}$

  Letters: $\Sigma = \{a,b, \ldots ,z\}$

- String: a finite, possibly empty sequence of symbols from an alphabet

  If $\Sigma = \{a,b,c\}$, valid strings include: a, ab, abc, aabbc

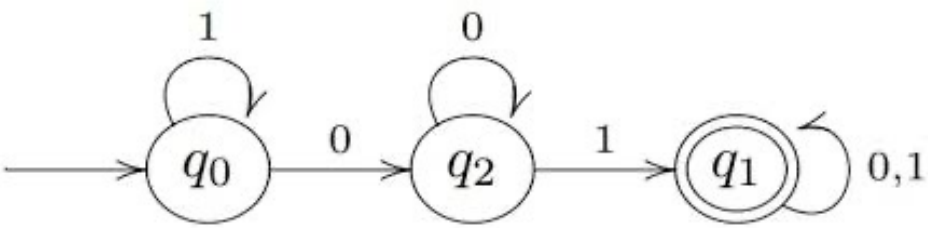
- Language: a set, often infinite, of strings

  $L_1 = \{ a, ab, bba \}$

  $L_2 = \{ a^n \mid n \geq 0 \}$

Theory of Automata and FL: A Quick Review

- Finite specifications of (possibly infinite) languages
  - Automaton – a recognizer; a machine that accepts all strings in a language (and rejects all other strings)



  - Grammar – a generator; a system for producing all strings in the language (and no other strings)

$$S \rightarrow a\ S\ b$$
$$S \rightarrow \varepsilon$$

Theory of Automata and FL: A Quick Review

- A particular language may be specified by many different grammars and automata
  - Many Grammars for One Language
  - Multiple Automata for One Language
    - DFA, NFA

- A grammar or automaton specifies only one language

Grammar	Language
$S \rightarrow a\ S\ b \mid \varepsilon$	$L = \{a^n b^n \mid n \geq 0\}$

Regular Expressions and Finite Automata

- The lexical grammar (structure) of most programming languages can be specified with regular expressions
  - Regular Expressions are a formal way to specify patterns i.e., Tokens Specification
- Tokens can be recognized by a Deterministic Finite Automaton (DFA)
  - Can be either table-driven or built by hand based on lexical grammar

Regular Expressions

- Defined over some alphabet Σ
- For programming languages, alphabet is usually ASCII or Unicode
- If RE is a regular expression, L(RE) is the language (set of strings) generated by RE

RE	L(RE)	Notes
a	{ a }	Singleton set, for each a in Σ
ε	{ ε }	Empty string
φ	{ }	Empty language

Regular Expressions: Basic Operations

RE	L(RE)	Notes
rs	L(r)L(s)	Concatenation
r|s	L(r) U L(s)	Combination (union)
r*	L(r)*	0 or more occurrences (Kleene closure)

- Precedence: *(highest), concatenation, |(lowest)
- Parentheses can be used to group REs as needed

Thank you!